

SIMATS ENGINEERING
ASSESSMENT TOOL 3 – MOCK INTERVIEW QUESTIONS (Low)

Course Name: Ethical Hacking	Code: ITA1404	CO: CO5
Duration: 90 min Date:	Total Marks:	Reg. No / Name : S Venkata Abhishek(192311053)

COURSE OUTCOME COVERED

CO5: *Analyze vulnerabilities in Windows and Linux systems and evaluate security weaknesses in messaging, web, and mobile applications using appropriate exploitation techniques and hacking tools. (BT5)*

1. NULL Sessions in Windows

NULL sessions allow unauthenticated users to enumerate information such as usernames, groups, and shared resources. I would first verify the exposure in an authorized environment using tools such as Nmap and Enum4linux, then determine what sensitive information is exposed and whether it enables further unauthorized access.

I would rate the risk based on the sensitivity of the information, lateral-movement potential, and business impact. In business terms, I would explain that anonymous access can help attackers map the organization and identify targets, increasing the risk of data theft and compromise.

2. Exposed RPC Services

I would identify the exposed RPC services and determine their versions, configurations, and network accessibility. Then I would compare them against known vulnerabilities and assess whether exploitation is reliable and within the engagement scope.

I would prioritize vulnerabilities that are remotely exploitable, highly impactful, reliable, and exposed to untrusted networks. I would avoid unnecessary exploitation if a safe validation method provides sufficient evidence.

3. SMB Foothold and Lateral Movement

After obtaining authorized access through SMB, I would evaluate available lateral-movement paths based on required privileges, network segmentation, credential availability, detection probability, and reliability.

Credential reuse, pass-the-hash, and remote execution have different risks. I would choose the least intrusive method that satisfies the testing objective and produces reliable evidence while minimizing operational impact and detection risk.

4. Weak Share Permissions

I would determine whether low-privileged users can read, modify, or execute sensitive files and whether writable locations are used by privileged processes.

I would validate the issue using harmless test files or controlled modifications rather than disrupting production data. The severity would depend on whether the weakness enables sensitive-data exposure, malware propagation, or privilege escalation.

5. SMBv1 in a Legacy Environment

SMBv1 is an outdated protocol with serious security weaknesses and a history of major attacks such as EternalBlue and WannaCry.

I would recommend migrating to SMBv2/SMBv3, but first identify legacy dependencies and test compatibility. Until migration is possible, compensating controls such as network segmentation, restricting SMB access, firewall filtering, patching, and monitoring should be implemented.

6. Windows Domain Reconnaissance

I would compare passive and active reconnaissance based on information quality, noise, detection probability, and business impact.

Passive or low-impact techniques should normally be preferred in production because they generate fewer alerts and operational risks. Active enumeration can be used when explicitly authorized and when more detailed information is required.

7. Low-Privilege Windows Account

I would evaluate privilege-escalation opportunities by reviewing system configuration, services, permissions, scheduled tasks, installed software, credentials, and security policies.

I would prioritize paths that are reliable, require minimal interaction, and have clear evidence of impact. However, the selected approach must remain within the engagement scope and avoid unnecessary system changes.

8. Suspected EternalBlue Vulnerability

I would not immediately exploit the suspected vulnerability. First, I would verify the operating-system version, SMB configuration, patch status, and vulnerability indicators using safe scanning and configuration checks.

Because exploitation can potentially destabilize vulnerable systems, I would validate carefully and obtain approval before any intrusive test. If the evidence confirms exposure, remediation should be prioritized immediately.

9. Linux Open Ports and Services

I would first identify the services, versions, configurations, and network exposure. I would then compare them against known vulnerabilities and determine whether they are necessary for business operations.

Internet-facing outdated services with known exploitable vulnerabilities receive the highest priority. Unknown ports should also be investigated because undocumented services increase the organization's attack surface.

10. Misconfigured NFS Shares

I would examine the NFS exports, permitted client networks, permissions, and root-squashing configuration.

I would safely demonstrate the issue by accessing only approved test files or non-sensitive resources. If the configuration allows access to sensitive data or creates a path to privilege escalation, I would classify it as high risk and recommend restricting exports and applying least privilege.

11. .rhosts and hosts.equiv

These mechanisms create trust relationships that may allow remote access without strong authentication. In modern environments, they are generally unnecessary and should be removed.

I would determine whether the files are active, which systems are trusted, and whether the trust relationship can actually be used. If they permit privileged unauthorized access, the risk is critical; if they are inactive remnants, the risk may be lower.

12. 777 File Permissions

Permissions of 777 allow everyone to read, write, and execute files or directories, creating significant security risks.

I would identify whether sensitive files or files executed by privileged processes have these permissions. If a low-privileged user can modify something executed with higher privileges, privilege escalation may be possible. Therefore, unnecessary 777 permissions should be corrected.

13. Linux Buffer Overflow

I would evaluate exploitability based on the vulnerable application's version, memory protections such as ASLR and stack canaries, required privileges, exploit complexity, and reliability.

A vulnerability may be technically serious but difficult to exploit in a particular environment. Therefore, I would distinguish between theoretical vulnerability and practical exploitability and prioritize remediation based on both impact and likelihood.

14. Initial Linux Access

After initial access, I would first perform controlled information gathering to understand the system, privileges, sensitive resources, and network position.

I would prioritize achieving the agreed testing objective rather than unnecessarily establishing persistence or moving laterally. Stealth and minimal system modification are important because excessive activity can affect production and trigger detection.

15. Linux Background Services

I would identify each service, determine its purpose, listening interfaces, configuration, and business dependency.

Unnecessary services should be disabled because they increase the attack surface. Required services should instead be patched, restricted, securely configured, and monitored.

The recommendation should always consider business requirements before disabling a service.

16. SQL Injection

I would first confirm the suspected SQL injection using safe, non-destructive testing. Then I would determine whether the vulnerability can affect authentication, database queries, or sensitive information.

Severity depends on exploitability, database privileges, accessible data, and business impact. If customer or financial data can be accessed, the vulnerability should receive a high or critical priority.

17. XSS Vulnerability

I would evaluate whether the XSS is stored or reflected, which users can trigger it, and whether sensitive actions or information can be affected.

For a client demonstration, I would use a harmless proof of concept that does not steal real credentials or data. The severity would depend on affected users, privileges, persistence, and potential account compromise.

18. Stored vs Reflected XSS

Stored XSS is generally more dangerous because the malicious content remains on the application and can affect multiple users without requiring the attacker to repeatedly deliver a link.

Reflected XSS normally requires user interaction, such as visiting a crafted URL.

Therefore, stored XSS is often considered higher risk, especially when it affects administrators or other privileged users.

19. CSRF

I would determine whether state-changing requests require unpredictable anti-CSRF tokens and whether appropriate cookie and origin protections are implemented.

I would test whether tokens are unique, unpredictable, properly validated, and tied to the user's session. I would also examine protections such as SameSite cookies and Origin/Referer validation.

If an attacker can perform sensitive actions without authorization, the vulnerability should receive higher priority.

20. Predictable Session IDs

I would evaluate whether session identifiers can be predicted or inferred by analyzing their randomness and lifecycle in a controlled test.

If an attacker can obtain another user's session identifier, session hijacking could result in account compromise without knowing the user's password.

For e-commerce or financial applications, the business impact can be severe because attackers may access personal information or perform unauthorized transactions.

ADVANCED SCENARIOS

21. SMBv1, EternalBlue and Weak Segmentation

SMBv1 creates significant risk because vulnerabilities such as EternalBlue can potentially allow remote compromise of vulnerable systems. Weak network segmentation increases the impact because compromised systems may communicate freely with critical systems.

I would evaluate patch status, SMBv1 exposure, network paths, and business dependencies before exploitation. Worm-like attacks could spread rapidly across vulnerable systems, while targeted attacks may focus on high-value systems.

The preferred strategy is to disable SMBv1 and migrate to SMBv2/3. Until migration is complete, segmentation, firewall restrictions, patching, access controls, and monitoring should be implemented.

22. Linux Multiple Services and Unknown Ports

I would first build an inventory of all listening services and identify their versions, configurations, ownership, and business purpose.

Outdated internet-facing services with known vulnerabilities should receive high priority. Unknown services are also important because administrators cannot properly secure what they cannot identify.

Automated scanners provide broad coverage, while manual verification confirms whether findings are genuine and business-relevant. Both approaches should be combined.

23. NFS Weak Permissions and No Logging

Unrestricted NFS shares can expose sensitive configuration files and may provide paths toward privilege escalation if writable or privileged resources are accessible.

I would safely validate access using approved test resources rather than modifying production files. The absence of logging increases the risk because unauthorized access may occur without adequate detection or forensic evidence.

Therefore, this should generally be treated as a **high-risk finding**, requiring restricted exports, least privilege, root squashing where appropriate, and centralized logging.

24. SQL Injection and Financial Data

SQL injection affecting an application containing financial data should generally be treated as a critical or high-priority vulnerability.

I would safely confirm injection and determine the potential database impact without extracting unnecessary sensitive information. Error-based injection may reveal information through database errors, union-based techniques may expose query results where applicable, and blind injection relies on application behavior rather than direct database output.

The primary remediation is parameterized queries and secure server-side input handling. Database accounts should also have least privilege, while monitoring and appropriate application-layer protections provide additional defense.

25. Windows Service Privilege Escalation

Services running as SYSTEM are highly sensitive. If low-privileged users can modify service executables or their directories, an attacker may potentially influence code executed with SYSTEM privileges.

I would safely verify service permissions, executable ownership, directory permissions, and service configuration without replacing production binaries. Path hijacking and binary replacement are examples of possible attack paths.

The preferred mitigation is to remove low-privilege write access, use secure service directories, enforce appropriate permissions and integrity controls, avoid unnecessary SYSTEM privileges, and monitor service configuration changes.